

Федеральное государственное автономное образовательное учреждение высшего
образования

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет информационных технологий

Кафедра «Информатика и информационные технологии»

Направление подготовки/ специальность: 09.03.02 Информационные системы и технологии

ОТЧЕТ

по проектной практике

Студент: Пронякин Роман Вадимович Группа: 251-331

Место прохождения практики: Московский Политех, кафедра «Информатика и
информационные технологии»

Отчет принят с оценкой _____ Дата _____

Руководитель практики: _____

Москва 2026

Оглавление

Введение	3
1. Общая информация о проекте.....	3
2. Общая характеристика деятельности организации (заказчика проекта) ..	4
3. Описание задания по проектной практике.....	4
3.1. Обязательная часть:.....	4
3.2. Вариативная часть:	4
4. Описание достигнутых результатов по проектной практике	5
4.1. Веб-сайты (Hugo).....	5
4.2. Документация в Markdown и Git.....	5
4.3. Дизайн-проект игры «Project SpaceWolf»	5
4.4. Вариативная часть: «Space Invaders».....	5
4.5. Личный вклад и приобретённые навыки	5
Заключение	6
Список использованной литературы	6
Приложения.....	6
Приложение А. Ссылка на репозиторий Github.....	6
Приложение Б. Скриншоты страниц веб-сайта проекта.....	7
Приложение В. Исходный код вариативной части.....	9
Приложение Г. Скриншот In-Game	25
Приложение Д. Скриншоты страниц веб-сайта Вариативной части.....	26

Введение

Целью проектной практики являлась разработка концепции и дизайна элементов космического шутера «Project SpaceWolf», а также создание веб-сайтов для презентации основного проекта и результатов вариативной части. В задачи практики входило: изучение референсов жанра on-rails shooter (серия StarFox), проектирование уникальных механик поведения противников, дизайн многоуровневых локаций, освоение генератора статических сайтов Hugo, работа с инструментами Git и Markdown, а также реализация классической игры «Space Invaders» на языке C++ с использованием OpenGL.

Практика проходила в период с 03 февраля 2026 г. по 24 мая 2026 г. под руководством куратора от кафедры «Информатика и информационные технологии».

1. Общая информация о проекте

Куратор проекта — Камозин Сергей Андреевич.

Название проекта:

«Project SpaceWolf» (космический шутер в жанре on-rails shooter).

Цели и задачи проекта:

Цель — создание динамичного аркадного симулятора космических сражений с проработанным визуальным стилем и уникальными тактическими задачами.

Задачи:

1. Разработка уникальных механик поведения противников-«головоломок».
2. Дизайн многоуровневых локаций с различными фазами (On-rails и All-range).
3. Создание атмосферного визуального стиля в духе Sci-Fi и киберпанка.
4. Интеграция системы тактических способностей игрока (Charge Shot, Barrel Roll).
5. Проектирование детализированных сценариев сражений с боссами и крупными объектами.

2. Общая характеристика деятельности организации (заказчика проекта)

Проект является инициативной студенческой разработкой в рамках дисциплины «Проектная деятельность» и находится на стадии активного прототипирования.

3. Описание задания по проектной практике

Задание состояло из **обязательной** (создание сайта-презентации и ведение документации) и **вариативной** (практическая реализация технологии из списка «Build Your Own X») частей.

3.1. Обязательная часть:

- Изучить синтаксис Markdown и вести всю документацию проекта в этом формате.
- Создать основной статический веб-сайт на базе Hugo, включающий:
 - домашнюю страницу с аннотацией проекта;
 - страницу «О проекте» с подробным описанием игровых особенностей;
 - страницу «Участники» с указанием вклада каждого члена команды;
 - страницу «Журнал» с записями о прогрессе разработки;
 - страницу «Ресурсы» со ссылками на использованные материалы и инструменты.
- Обеспечить публикацию сайтов через GitHub Pages.

3.2. Вариативная часть:

- Выбрать проект из списка «Build Your Own X» и реализовать его.
- Мною был выбран проект: «**Space Invaders from Scratch**» на языке C++.
- Реализовать низкоуровневый рендеринг через OpenGL без использования готовых игровых движков.
- Создать отдельный веб-сайт (также на Hugo) для описания процесса реализации вариативной части.

4. Описание достигнутых результатов по проектной практике

4.1. Веб-сайты (Hugo)

- Созданы два статических сайта: основной (site/main-site) и вариативный (site/var-site).
- В качестве основы использован генератор Hugo и тема Ananke.
- Контент сайтов структурирован согласно заданию (разделы «О проекте», «Участники», «Журнал», «Ресурсы»).
- Настроена автоматическая публикация через GitHub.

4.2. Документация в Markdown и Git

- Создан структурированный репозиторий с файлами: README.md, project-info.md, docs/main-report.md, docs/variative-report.md, docs/byox-guide.md.
- Настроена система контроля версий Git, все изменения фиксировались осмысленными коммитами.

4.3. Дизайн-проект игры «Project SpaceWolf»

- Разработана концепция тактических противников, каждый из которых требует особого подхода (Корабль «Скат», Тентакль, Рой, Тандемные двойники, Бутон-снайпер).
- Спроектированы уровни 2 («Космическая станция») и 3 («Древний узел связи»), включающие пролет через кладбище кораблей и сражения с боссами в узких желобах (Trench Runs).
- Внедрены элементы визуального стиля в духе «Бегущего по лезвию 2049».

4.4. Вариативная часть: «Space Invaders»

- Реализован программный рендеринг (CPU-буфер) с выводом через текстуру OpenGL.
- Написаны шейдеры для отображения игрового поля.
- Реализована физика столкновений (AABB), система анимации пришельцев и текстовый интерфейс.
- Использован стек технологий: C++17, CMake, GLFW, GLEW.

4.5. Личный вклад и приобретённые навыки

- **Геймдизайн** — научился проектировать врагов как «тактические задачи» и выстраивать геометрию уровней для on-rails шутера.

- **C++ и OpenGL** — освоил низкоуровневую работу с графикой, управление памятью кадрового буфера и написание шейдеров.
- **Веб-технологии** — изучил работу с генератором статических сайтов Hugo, настройку тем и структуру шаблонов.
- **Markdown и Git** — закрепил навыки ведения технической документации и командной работы через Git.

Заключение

За время проектной практики я полностью выполнил поставленное задание. Мною были разработаны ключевые элементы геймдизайна для проекта «Project SpaceWolf», созданы презентационные сайты и реализована технически сложная вариативная часть в виде игры на C++/OpenGL.

Практика позволила мне не только применить теоретические знания в области программирования и дизайна, но и освоить современные инструменты промышленной разработки (Git, CMake, Hugo). Разработанные концепции уровней и противников легли в основу дальнейшего развития игрового проекта.

Список использованной литературы

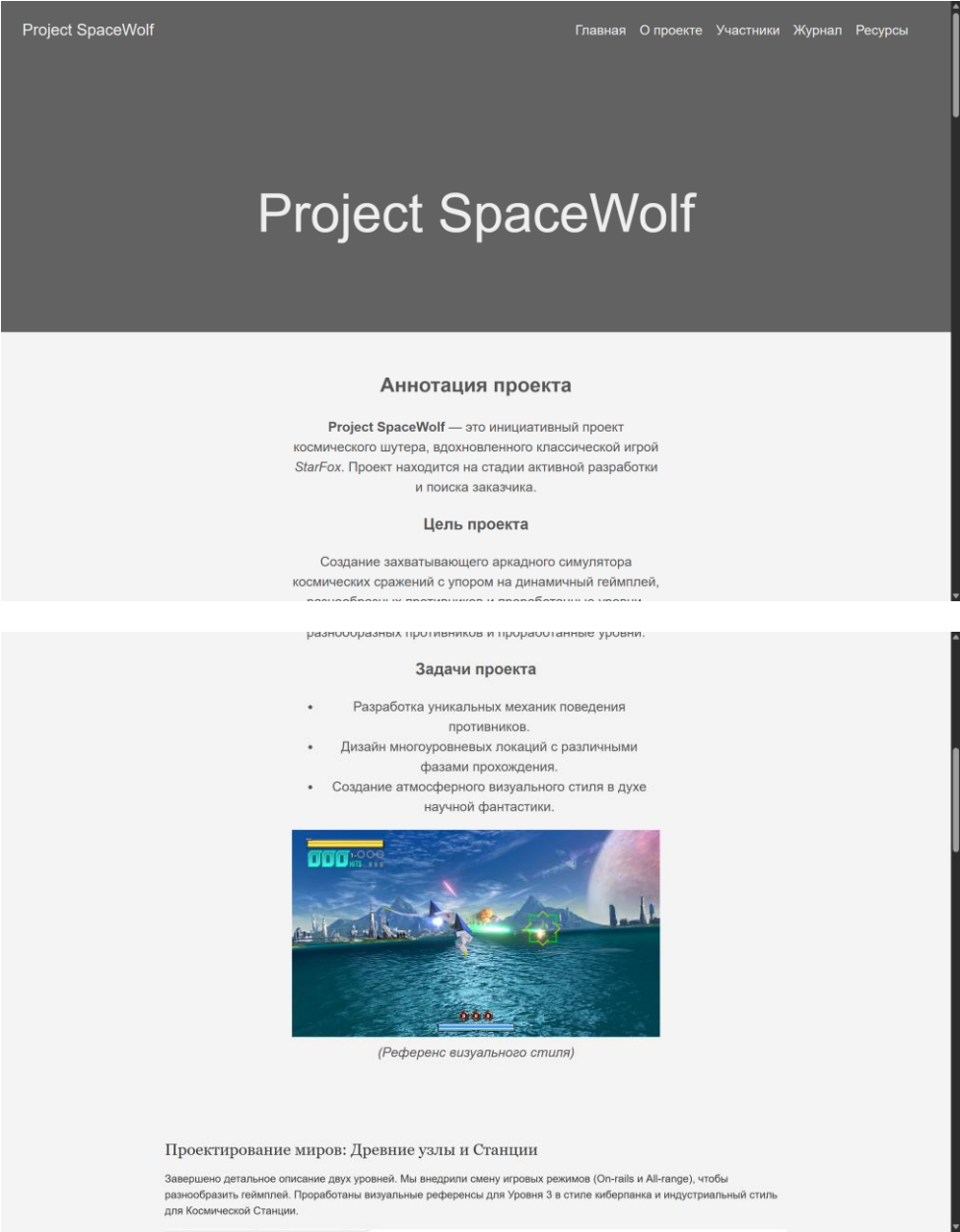
1. Официальная документация Hugo: gohugo.io/documentation
2. OpenGL Graphics Library Documentation: opengl.org/documentation
3. Справочник по C++ (cppreference): cppreference.com
4. Руководство «Space Invaders from Scratch» (Nick Tasios): nicktasios.nl
5. Документация Markdown: markdownguide.org
6. StarFox Wiki (референсы жанра): starfoxwiki.info

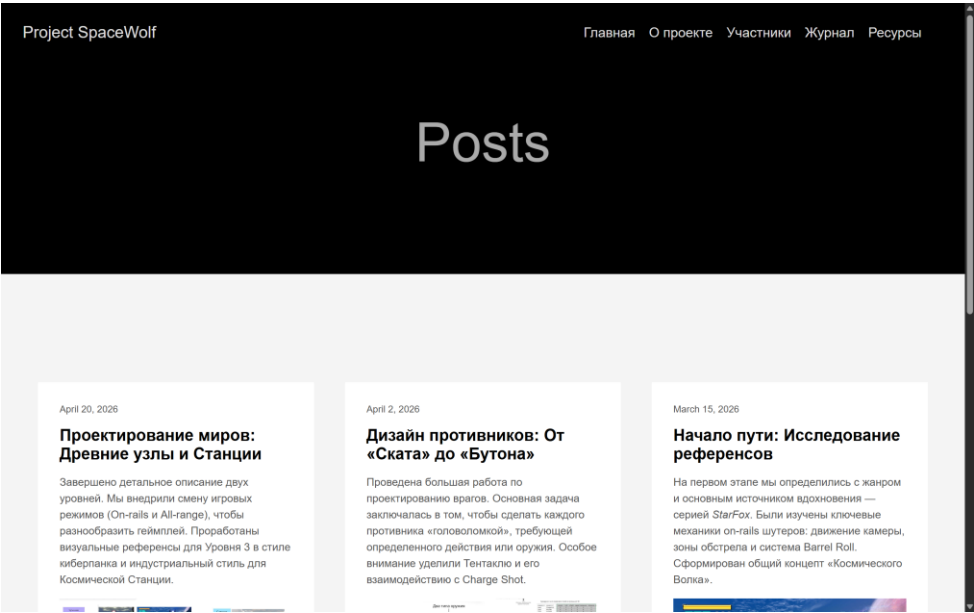
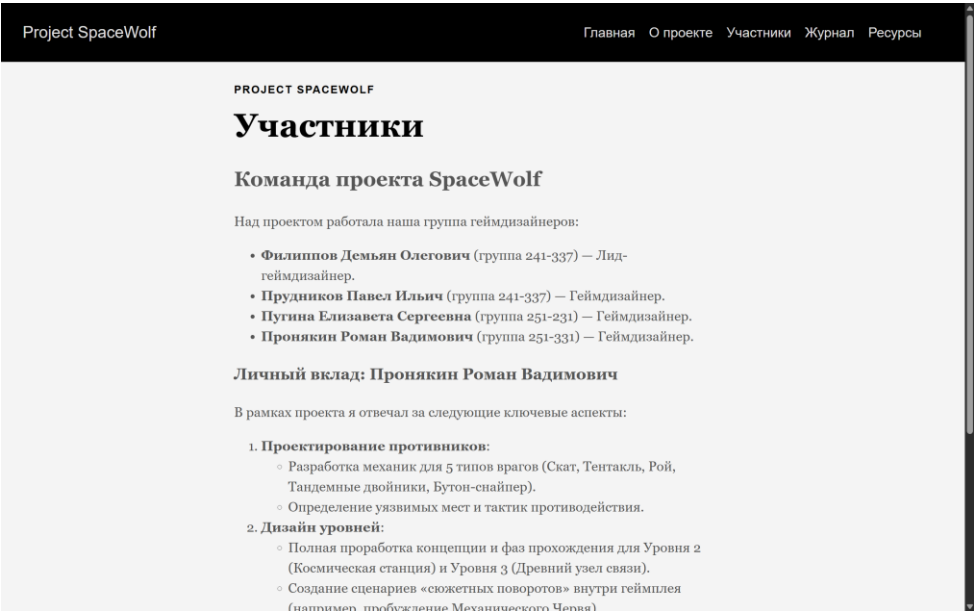
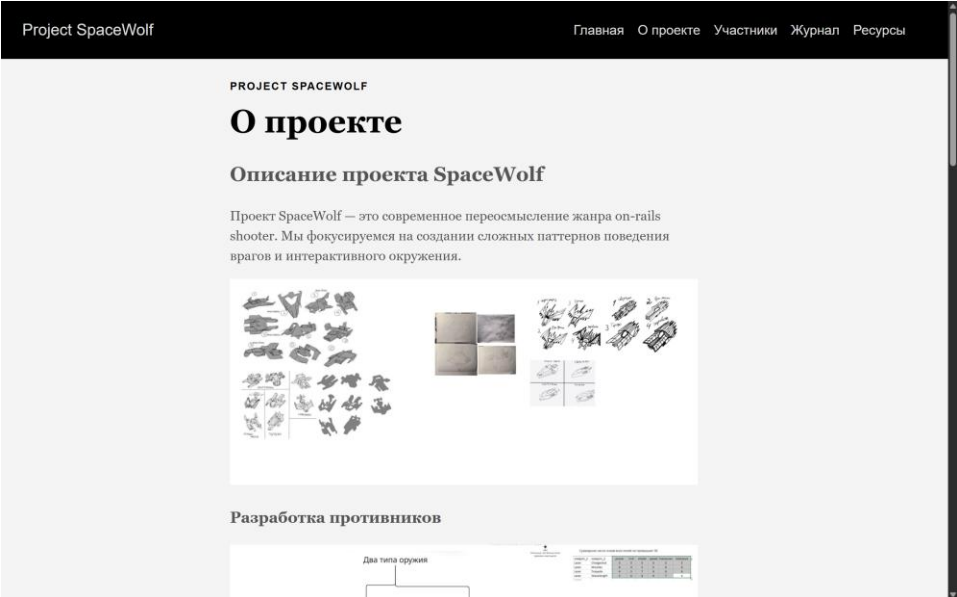
Приложения

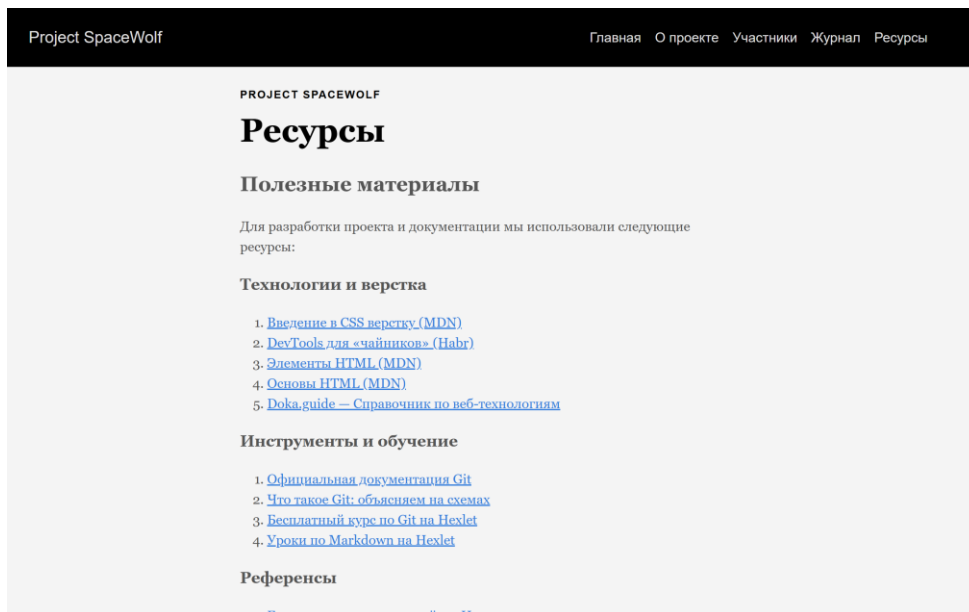
Приложение А. Ссылка на репозиторий Github

https://github.com/h3lv0k/Practice_project_2026

Приложение Б. Скриншоты страниц веб-сайта проекта







Приложение В. Исходный код вариативной части

```
#include <GL/glew.h>
#include <GLFW/glfw3.h>

#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <algorithm>
#include <vector>

namespace {
constexpr int kBufferWidth = 640;
constexpr int kBufferHeight = 480;
constexpr int kAlienRows = 5;
constexpr int kAlienCols = 11;
constexpr int kAlienCount = kAlienRows * kAlienCols;
constexpr int kMaxBullets = 128;
constexpr int kPlayerSpeed = 3;
constexpr int kBulletSpeed = 4;
constexpr int kAlienAnimSpeed = 30;
constexpr int kAlienMoveDelay = 20;
constexpr int kAlienStep = 4;
constexpr int kAlienDrop = 12;
constexpr int kDeathFrames = 10;

struct Buffer {
    int width;
    int height;
    std::vector<uint32_t> data;
};

struct Sprite {
```

```
    int width;
    int height;
    const uint8_t* data;
};

enum AlienType : uint8_t {
    ALIEN_NONE = 0,
    ALIEN_A = 1,
    ALIEN_B = 2,
    ALIEN_C = 3
};

struct Alien {
    int x;
    int y;
    AlienType type;
    int death_timer;
};

struct Player {
    int x;
    int y;
    int life;
};

struct Bullet {
    int x;
    int y;
    int dir;
    bool active;
};

struct Game {
    Alien aliens[kAlienCount];
    Bullet bullets[kMaxBullets];
    Player player;
    int score;
    int anim_tick;
    int alien_dir;
    int alien_move_tick;
};

struct InputState {
    bool left;
    bool right;
    bool fire_requested;
};

InputState g_input{};
```

```

uint32_t rgba_to_u32(uint8_t r, uint8_t g, uint8_t b, uint8_t a = 255) {
    return static_cast<uint32_t>(r) | (static_cast<uint32_t>(g) << 8) |
           (static_cast<uint32_t>(b) << 16) | (static_cast<uint32_t>(a) << 24);
}

void buffer_clear(Buffer& buffer, uint32_t color) {
    std::fill(buffer.data.begin(), buffer.data.end(), color);
}

void buffer_draw_sprite(Buffer& buffer, const Sprite& sprite, int x, int y, uint32_t color) {
    for (int sy = 0; sy < sprite.height; ++sy) {
        int by = y + sy;
        if (by < 0 || by >= buffer.height) {
            continue;
        }
        for (int sx = 0; sx < sprite.width; ++sx) {
            int bx = x + sx;
            if (bx < 0 || bx >= buffer.width) {
                continue;
            }
            if (sprite.data[sy * sprite.width + sx] == 0) {
                continue;
            }
            buffer.data[by * buffer.width + bx] = color;
        }
    }
}

void buffer_draw_hline(Buffer& buffer, int x, int y, int w, uint32_t color) {
    if (y < 0 || y >= buffer.height) {
        return;
    }
    int start = x < 0 ? 0 : x;
    int end = x + w;
    if (end > buffer.width) {
        end = buffer.width;
    }
    for (int bx = start; bx < end; ++bx) {
        buffer.data[y * buffer.width + bx] = color;
    }
}

bool sprite_overlap(int ax, int ay, const Sprite& a, int bx, int by, const Sprite& b) {
    int left_a = ax;
    int right_a = ax + a.width;
    int top_a = ay;
    int bottom_a = ay + a.height;

    int left_b = bx;
    int right_b = bx + b.width;

```

```

    int top_b = by;
    int bottom_b = by + b.height;

    if (left_a >= right_b || left_b >= right_a) {
        return false;
    }
    if (top_a >= bottom_b || top_b >= bottom_a) {
        return false;
    }
    return true;
}

void error_callback(int error, const char* description) {
    std::fprintf(stderr, "GLFW error %d: %s\n", error, description);
}

void key_callback(GLFWwindow* window, int key, int scancode, int action, int mods) {
    (void)scancode;
    (void)mods;

    if (key == GLFW_KEY_ESCAPE && action == GLFW_PRESS) {
        glfwSetWindowShouldClose(window, GLFW_TRUE);
        return;
    }

    if (key == GLFW_KEY_LEFT) {
        g_input.left = (action != GLFW_RELEASE);
    } else if (key == GLFW_KEY_RIGHT) {
        g_input.right = (action != GLFW_RELEASE);
    } else if (key == GLFW_KEY_SPACE) {
        if (action == GLFW_RELEASE) {
            g_input.fire_requested = true;
        }
    }
}

void build_sprite_from_strings(const char* const* rows, int width, int height, uint8_t* out) {
    for (int y = 0; y < height; ++y) {
        for (int x = 0; x < width; ++x) {
            char c = rows[y][x];
            out[y * width + x] = (c == 'X') ? 1 : 0;
        }
    }
}

// 5x7 ASCII font for characters 32-126.
static const uint8_t kFont5x7[95][5] = {
    {0x00, 0x00, 0x00, 0x00, 0x00}, // Space
    {0x00, 0x00, 0x5F, 0x00, 0x00}, // !
    {0x00, 0x07, 0x00, 0x07, 0x00}, // "

```

```
{0x14, 0x7F, 0x14, 0x7F, 0x14}, // #
{0x24, 0x2A, 0x7F, 0x2A, 0x12}, // $
{0x23, 0x13, 0x08, 0x64, 0x62}, // %
{0x36, 0x49, 0x55, 0x22, 0x50}, // &
{0x00, 0x05, 0x03, 0x00, 0x00}, // '
{0x00, 0x1C, 0x22, 0x41, 0x00}, // (
{0x00, 0x41, 0x22, 0x1C, 0x00}, // )
{0x14, 0x08, 0x3E, 0x08, 0x14}, // *
{0x08, 0x08, 0x3E, 0x08, 0x08}, // +
{0x00, 0x50, 0x30, 0x00, 0x00}, // ,
{0x08, 0x08, 0x08, 0x08, 0x08}, // -
{0x00, 0x60, 0x60, 0x00, 0x00}, // .
{0x20, 0x10, 0x08, 0x04, 0x02}, // /
{0x3E, 0x51, 0x49, 0x45, 0x3E}, // 0
{0x00, 0x42, 0x7F, 0x40, 0x00}, // 1
{0x42, 0x61, 0x51, 0x49, 0x46}, // 2
{0x21, 0x41, 0x45, 0x4B, 0x31}, // 3
{0x18, 0x14, 0x12, 0x7F, 0x10}, // 4
{0x27, 0x45, 0x45, 0x45, 0x39}, // 5
{0x3C, 0x4A, 0x49, 0x49, 0x30}, // 6
{0x01, 0x71, 0x09, 0x05, 0x03}, // 7
{0x36, 0x49, 0x49, 0x49, 0x36}, // 8
{0x06, 0x49, 0x49, 0x29, 0x1E}, // 9
{0x00, 0x36, 0x36, 0x00, 0x00}, // :
{0x00, 0x56, 0x36, 0x00, 0x00}, // ;
{0x08, 0x14, 0x22, 0x41, 0x00}, // <
{0x14, 0x14, 0x14, 0x14, 0x14}, // =
{0x00, 0x41, 0x22, 0x14, 0x08}, // >
{0x02, 0x01, 0x51, 0x09, 0x06}, // ?
{0x32, 0x49, 0x79, 0x41, 0x3E}, // @
{0x7E, 0x11, 0x11, 0x11, 0x7E}, // A
{0x7F, 0x49, 0x49, 0x49, 0x36}, // B
{0x3E, 0x41, 0x41, 0x41, 0x22}, // C
{0x7F, 0x41, 0x41, 0x22, 0x1C}, // D
{0x7F, 0x49, 0x49, 0x49, 0x41}, // E
{0x7F, 0x09, 0x09, 0x09, 0x01}, // F
{0x3E, 0x41, 0x49, 0x49, 0x7A}, // G
{0x7F, 0x08, 0x08, 0x08, 0x7F}, // H
{0x00, 0x41, 0x7F, 0x41, 0x00}, // I
{0x20, 0x40, 0x41, 0x3F, 0x01}, // J
{0x7F, 0x08, 0x14, 0x22, 0x41}, // K
{0x7F, 0x40, 0x40, 0x40, 0x40}, // L
{0x7F, 0x02, 0x0C, 0x02, 0x7F}, // M
{0x7F, 0x04, 0x08, 0x10, 0x7F}, // N
{0x3E, 0x41, 0x41, 0x41, 0x3E}, // O
{0x7F, 0x09, 0x09, 0x09, 0x06}, // P
{0x3E, 0x41, 0x51, 0x21, 0x5E}, // Q
{0x7F, 0x09, 0x19, 0x29, 0x46}, // R
{0x46, 0x49, 0x49, 0x49, 0x31}, // S
{0x01, 0x01, 0x7F, 0x01, 0x01}, // T
```

```

    {0x3F, 0x40, 0x40, 0x40, 0x3F}, // U
    {0x1F, 0x20, 0x40, 0x20, 0x1F}, // V
    {0x7F, 0x20, 0x18, 0x20, 0x7F}, // W
    {0x63, 0x14, 0x08, 0x14, 0x63}, // X
    {0x03, 0x04, 0x78, 0x04, 0x03}, // Y
    {0x61, 0x51, 0x49, 0x45, 0x43}, // Z
    {0x00, 0x7F, 0x41, 0x41, 0x00}, // [
    {0x02, 0x04, 0x08, 0x10, 0x20}, // \
    {0x00, 0x41, 0x41, 0x7F, 0x00}, // ]
    {0x04, 0x02, 0x01, 0x02, 0x04}, // ^
    {0x40, 0x40, 0x40, 0x40, 0x40}, // _
    {0x00, 0x01, 0x02, 0x04, 0x00}, // `
    {0x20, 0x54, 0x54, 0x54, 0x78}, // a
    {0x7F, 0x48, 0x44, 0x44, 0x38}, // b
    {0x38, 0x44, 0x44, 0x44, 0x20}, // c
    {0x38, 0x44, 0x44, 0x48, 0x7F}, // d
    {0x38, 0x54, 0x54, 0x54, 0x18}, // e
    {0x08, 0x7E, 0x09, 0x01, 0x02}, // f
    {0x0C, 0x52, 0x52, 0x52, 0x3E}, // g
    {0x7F, 0x08, 0x04, 0x04, 0x78}, // h
    {0x00, 0x44, 0x7D, 0x40, 0x00}, // i
    {0x20, 0x40, 0x44, 0x3D, 0x00}, // j
    {0x7F, 0x10, 0x28, 0x44, 0x00}, // k
    {0x00, 0x41, 0x7F, 0x40, 0x00}, // l
    {0x7C, 0x04, 0x18, 0x04, 0x78}, // m
    {0x7C, 0x08, 0x04, 0x04, 0x78}, // n
    {0x38, 0x44, 0x44, 0x44, 0x38}, // o
    {0x7C, 0x14, 0x14, 0x14, 0x08}, // p
    {0x08, 0x14, 0x14, 0x18, 0x7C}, // q
    {0x7C, 0x08, 0x04, 0x04, 0x08}, // r
    {0x48, 0x54, 0x54, 0x54, 0x20}, // s
    {0x04, 0x3F, 0x44, 0x40, 0x20}, // t
    {0x3C, 0x40, 0x40, 0x20, 0x7C}, // u
    {0x1C, 0x20, 0x40, 0x20, 0x1C}, // v
    {0x3C, 0x40, 0x30, 0x40, 0x3C}, // w
    {0x44, 0x28, 0x10, 0x28, 0x44}, // x
    {0x0C, 0x50, 0x50, 0x50, 0x3C}, // y
    {0x44, 0x64, 0x54, 0x4C, 0x44}, // z
    {0x00, 0x08, 0x36, 0x41, 0x00}, // {
    {0x00, 0x00, 0x7F, 0x00, 0x00}, // |
    {0x00, 0x41, 0x36, 0x08, 0x00}, // }
    {0x08, 0x04, 0x08, 0x10, 0x08} // ~
};

```

```

void buffer_draw_char(Buffer& buffer, char c, int x, int y, uint32_t color) {
    if (c < 32 || c > 126) {
        return;
    }
    const uint8_t* glyph = kFont5x7[c - 32];
    for (int col = 0; col < 5; ++col) {

```

```

        uint8_t bits = glyph[col];
        for (int row = 0; row < 7; ++row) {
            if (bits & (1 << row)) {
                int px = x + col;
                int py = y + row;
                if (px >= 0 && px < buffer.width && py >= 0 && py < buffer.height) {
                    buffer.data[py * buffer.width + px] = color;
                }
            }
        }
    }
}

void buffer_draw_text(Buffer& buffer, const char* text, int x, int y, uint32_t color) {
    int cursor = x;
    for (const char* p = text; *p; ++p) {
        buffer_draw_char(buffer, *p, cursor, y, color);
        cursor += 6;
    }
}

void buffer_draw_number(Buffer& buffer, int value, int x, int y, uint32_t color) {
    char buf[16];
    std::snprintf(buf, sizeof(buf), "%d", value);
    buffer_draw_text(buffer, buf, x, y, color);
}

static uint8_t kAlienA0[11 * 8];
static uint8_t kAlienA1[11 * 8];
static uint8_t kAlienB0[11 * 8];
static uint8_t kAlienB1[11 * 8];
static uint8_t kAlienC0[11 * 8];
static uint8_t kAlienC1[11 * 8];
static uint8_t kAlienDead[11 * 8];
static uint8_t kPlayerSprite[13 * 8];
static uint8_t kBulletSprite[1 * 4];

Sprite g_alien_a_0{11, 8, kAlienA0};
Sprite g_alien_a_1{11, 8, kAlienA1};
Sprite g_alien_b_0{11, 8, kAlienB0};
Sprite g_alien_b_1{11, 8, kAlienB1};
Sprite g_alien_c_0{11, 8, kAlienC0};
Sprite g_alien_c_1{11, 8, kAlienC1};
Sprite g_alien_dead{11, 8, kAlienDead};
Sprite g_player{13, 8, kPlayerSprite};
Sprite g_bullet{1, 4, kBulletSprite};

void init_sprites() {
    const char* alien_a_0[8] = {
        "..X.....X..",

```

```

        "...XXXXX...",
        "..XXXXXXXX..",
        ".XX.XXX.XX.",
        "XXXXXXXXXXXX",
        "X.XXXXXX.X",
        "X.X....X.X",
        "...XX.XX..."
    };

    const char* alien_a_1[8] = {
        "..X....X..",
        "...XXXXX...",
        "..XXXXXXXX..",
        ".XX.XXX.XX.",
        "XXXXXXXXXXXX",
        ".X.XXXXX.X.",
        "..X....X..",
        ".X..XX.XX."
    };

    const char* alien_b_0[8] = {
        "...XXXXX...",
        "..XXXXXXXX..",
        ".XX.XXX.XX.",
        "XXXXXXXXXXXX",
        "XXX....XXX",
        "XX.XXXXX.XX",
        "X.X....X.X",
        "..XX...XX.."
    };

    const char* alien_b_1[8] = {
        "...XXXXX...",
        "..XXXXXXXX..",
        ".XX.XXX.XX.",
        "XXXXXXXXXXXX",
        "XX.....XX",
        "X.XXXXXX.X",
        "..X....X..",
        ".X..XX.XX."
    };

    const char* alien_c_0[8] = {
        "...XX.XX...",
        "..XXXXXXXX.",
        ".XXXXXXXXX.",
        ".XXX.XXX.XX",
        "XXXXXXXXXXXX",
        "X.XX.XX.XX.",
        "X.....X",
        "..XX...XX.."
    };

    const char* alien_c_1[8] = {
        "...XX.XX...",

```



```

        "..XXXXXXXX.",
        ".XXXXXXXXX.",
        ".XXX.XXX.XX",
        "XXXXXXXXXXXX",
        ".XX.XX.XX.X",
        "..X....X..",
        ".X..XX..X."
    };

    const char* alien_dead[8] = {
        ".....",
        "...X...X...",
        "....X.X....",
        "XXXXX.XXXXX",
        "....X.X....",
        "...X...X...",
        ".....",
        "....."
    };

    const char* player[8] = {
        "....XXXXX....",
        "...XXXXXXX...",
        "..XXXXXXXXX..",
        ".XXX.XXX.XXX.",
        "XXXXXXXXXXXXXX",
        "XXXX..X..XXXX",
        "XXX.....XXX",
        "XX.....XX"
    };

    const char* bullet[4] = {
        "X",
        "X",
        "X",
        "X"
    };

    build_sprite_from_strings(alien_a_0, 11, 8, kAlienA0);
    build_sprite_from_strings(alien_a_1, 11, 8, kAlienA1);
    build_sprite_from_strings(alien_b_0, 11, 8, kAlienB0);
    build_sprite_from_strings(alien_b_1, 11, 8, kAlienB1);
    build_sprite_from_strings(alien_c_0, 11, 8, kAlienC0);
    build_sprite_from_strings(alien_c_1, 11, 8, kAlienC1);
    build_sprite_from_strings(alien_dead, 11, 8, kAlienDead);
    build_sprite_from_strings(player, 13, 8, kPlayerSprite);
    build_sprite_from_strings(bullet, 1, 4, kBulletSprite);
}

const Sprite& alien_frame(AlienType type, int anim_tick) {
    bool toggle = (anim_tick / kAlienAnimSpeed) % 2 == 0;
    switch (type) {
        case ALIEN_A:

```

```

        return toggle ? g_alien_a_0 : g_alien_a_1;
    case ALIEN_B:
        return toggle ? g_alien_b_0 : g_alien_b_1;
    case ALIEN_C:
        return toggle ? g_alien_c_0 : g_alien_c_1;
    default:
        return g_alien_a_0;
    }
}

int score_for_type(AlienType type) {
    switch (type) {
        case ALIEN_A:
            return 40;
        case ALIEN_B:
            return 20;
        case ALIEN_C:
            return 10;
        default:
            return 0;
    }
}

void init_game(Game& game) {
    game.score = 0;
    game.anim_tick = 0;
    game.alien_dir = 1;
    game.alien_move_tick = 0;
    game.player.life = 3;
    game.player.x = (kBufferWidth / 2) - (g_player.width / 2);
    game.player.y = kBufferHeight - 40;

    for (int i = 0; i < kMaxBullets; ++i) {
        game.bullets[i].active = false;
    }

    int start_x = 80;
    int start_y = 80;
    int spacing_x = 32;
    int spacing_y = 24;

    int index = 0;
    for (int row = 0; row < kAlienRows; ++row) {
        for (int col = 0; col < kAlienCols; ++col) {
            Alien& alien = game.aliens[index++];
            alien.x = start_x + col * spacing_x;
            alien.y = start_y + row * spacing_y;
            alien.death_timer = 0;
            if (row == 0) {
                alien.type = ALIEN_A;
            }
        }
    }
}

```

```

        } else if (row <= 2) {
            alien.type = ALIEN_B;
        } else {
            alien.type = ALIEN_C;
        }
    }
}

void spawn_bullet(Game& game, int x, int y, int dir) {
    for (int i = 0; i < kMaxBullets; ++i) {
        Bullet& bullet = game.bullets[i];
        if (!bullet.active) {
            bullet.active = true;
            bullet.x = x;
            bullet.y = y;
            bullet.dir = dir;
            return;
        }
    }
}

void update_game(Game& game) {
    game.anim_tick++;
    game.alien_move_tick++;

    int vx = 0;
    if (g_input.left) {
        vx -= kPlayerSpeed;
    }
    if (g_input.right) {
        vx += kPlayerSpeed;
    }
    game.player.x += vx;
    if (game.player.x < 0) {
        game.player.x = 0;
    }
    if (game.player.x + g_player.width >= kBufferWidth) {
        game.player.x = kBufferWidth - g_player.width;
    }

    if (g_input.fire_requested) {
        g_input.fire_requested = false;
        int bx = game.player.x + (g_player.width / 2);
        int by = game.player.y - g_bullet.height;
        spawn_bullet(game, bx, by, -kBulletSpeed);
    }

    if (game.alien_move_tick >= kAlienMoveDelay) {
        game.alien_move_tick = 0;
    }
}

```

```

int left = kBufferWidth;
int right = 0;
bool any = false;

for (int i = 0; i < kAlienCount; ++i) {
    const Alien& alien = game.aliens[i];
    if (alien.type == ALIEN_NONE) {
        continue;
    }
    const Sprite& sprite = alien_frame(alien.type, game.anim_tick);
    left = std::min(left, alien.x);
    right = std::max(right, alien.x + sprite.width);
    any = true;
}

if (any) {
    int next_left = left + game.alien_dir * kAlienStep;
    int next_right = right + game.alien_dir * kAlienStep;
    bool hit_edge = next_left < 10 || next_right > (kBufferWidth - 10);

    if (hit_edge) {
        game.alien_dir = -game.alien_dir;
        for (int i = 0; i < kAlienCount; ++i) {
            Alien& alien = game.aliens[i];
            if (alien.type == ALIEN_NONE) {
                continue;
            }
            alien.y += kAlienDrop;
        }
    } else {
        for (int i = 0; i < kAlienCount; ++i) {
            Alien& alien = game.aliens[i];
            if (alien.type == ALIEN_NONE) {
                continue;
            }
            alien.x += game.alien_dir * kAlienStep;
        }
    }
}

for (int i = 0; i < kMaxBullets; ++i) {
    Bullet& bullet = game.bullets[i];
    if (!bullet.active) {
        continue;
    }
    bullet.y += bullet.dir;
    if (bullet.y < 0 || bullet.y > kBufferHeight) {
        bullet.active = false;
    }
}

```

```

    }

    for (int i = 0; i < kMaxBullets; ++i) {
        Bullet& bullet = game.bullets[i];
        if (!bullet.active) {
            continue;
        }
        for (int a = 0; a < kAlienCount; ++a) {
            Alien& alien = game.aliens[a];
            if (alien.type == ALIEN_NONE) {
                continue;
            }
            const Sprite& sprite = alien_frame(alien.type, game.anim_tick);
            if (sprite_overlap(bullet.x, bullet.y, g_bullet, alien.x, alien.y, sprite)) {
                bullet.active = false;
                alien.death_timer = kDeathFrames;
                game.score += score_for_type(alien.type);
                alien.type = ALIEN_NONE;
                break;
            }
        }
    }
}

for (int a = 0; a < kAlienCount; ++a) {
    Alien& alien = game.aliens[a];
    if (alien.death_timer > 0) {
        alien.death_timer--;
    }
}
}

void render_game(Buffer& buffer, const Game& game) {
    uint32_t black = rgba_to_u32(0, 0, 0);
    uint32_t green = rgba_to_u32(40, 220, 40);
    uint32_t white = rgba_to_u32(230, 230, 230);
    uint32_t red = rgba_to_u32(230, 60, 60);

    buffer_clear(buffer, black);

    buffer_draw_text(buffer, "SCORE", 20, 10, white);
    buffer_draw_number(buffer, game.score, 80, 10, white);
    buffer_draw_text(buffer, "CREDIT 00", kBufferWidth - 110, 10, white);
    buffer_draw_hline(buffer, 10, 30, kBufferWidth - 20, white);

    for (int i = 0; i < kAlienCount; ++i) {
        const Alien& alien = game.aliens[i];
        if (alien.type != ALIEN_NONE) {
            const Sprite& sprite = alien_frame(alien.type, game.anim_tick);
            buffer_draw_sprite(buffer, sprite, alien.x, alien.y, green);
        } else if (alien.death_timer > 0) {

```

```

        buffer_draw_sprite(buffer, g_alien_dead, alien.x, alien.y, red);
    }
}

buffer_draw_sprite(buffer, g_player, game.player.x, game.player.y, green);

for (int i = 0; i < kMaxBullets; ++i) {
    const Bullet& bullet = game.bullets[i];
    if (!bullet.active) {
        continue;
    }
    buffer_draw_sprite(buffer, g_bullet, bullet.x, bullet.y, white);
}
}

GLuint compile_shader(GLenum type, const char* source) {
    GLuint shader = glCreateShader(type);
    glShaderSource(shader, 1, &source, nullptr);
    glCompileShader(shader);

    GLint ok = 0;
    glGetShaderiv(shader, GL_COMPILE_STATUS, &ok);
    if (!ok) {
        char log[1024];
        glGetShaderInfoLog(shader, sizeof(log), nullptr, log);
        std::fprintf(stderr, "Shader compile error: %s\n", log);
    }
    return shader;
}

GLuint create_program() {
    const char* vertex_src =
        "#version 330 core\n"
        "out vec2 v_uv;\n"
        "void main() {\n"
        "    vec2 pos = vec2(-1.0, -1.0);\n"
        "    if (gl_VertexID == 1) pos = vec2(3.0, -1.0);\n"
        "    if (gl_VertexID == 2) pos = vec2(-1.0, 3.0);\n"
        "    v_uv = pos * 0.5 + 0.5;\n"
        "    gl_Position = vec4(pos, 0.0, 1.0);\n"
        "}\n";

    const char* fragment_src =
        "#version 330 core\n"
        "in vec2 v_uv;\n"
        "out vec4 fragColor;\n"
        "uniform sampler2D u_texture;\n"
        "void main() {\n"
        "    vec2 uv = vec2(v_uv.x, 1.0 - v_uv.y);\n"
        "    fragColor = texture(u_texture, uv);\n"

```

```

        "}\n";

GLuint vs = compile_shader(GL_VERTEX_SHADER, vertex_src);
GLuint fs = compile_shader(GL_FRAGMENT_SHADER, fragment_src);

GLuint program = glCreateProgram();
glAttachShader(program, vs);
glAttachShader(program, fs);
glLinkProgram(program);

GLint ok = 0;
glGetProgramiv(program, GL_LINK_STATUS, &ok);
if (!ok) {
    char log[1024];
    glGetProgramInfoLog(program, sizeof(log), nullptr, log);
    std::fprintf(stderr, "Program link error: %s\n", log);
}

glDeleteShader(vs);
glDeleteShader(fs);
return program;
}

} // namespace

int main() {
    glfwSetErrorCallback(error_callback);
    if (!glfwInit()) {
        return 1;
    }

    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);

    GLFWwindow* window = glfwCreateWindow(kBufferWidth, kBufferHeight, "Space Invaders", nullptr, nullptr);
    if (!window) {
        glfwTerminate();
        return 1;
    }

    glfwMakeContextCurrent(window);
    glfwSwapInterval(1);
    glfwSetKeyCallback(window, key_callback);

    glewExperimental = GL_TRUE;
    if (glewInit() != GLEW_OK) {
        std::fprintf(stderr, "Failed to initialize GLEW\n");
        glfwDestroyWindow(window);
        glfwTerminate();
    }
}

```

```

        return 1;
    }

    Buffer buffer{kBufferWidth, kBufferHeight, std::vector<uint32_t>(kBufferWidth * kBufferHeight)};
    init_sprites();

    GLuint vao = 0;
    glGenVertexArrays(1, &vao);
    glBindVertexArray(vao);

    GLuint program = create_program();
    glUseProgram(program);

    GLuint texture = 0;
    glGenTextures(1, &texture);
    glBindTexture(GL_TEXTURE_2D, texture);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, buffer.width, buffer.height, 0,
                 GL_RGBA, GL_UNSIGNED_BYTE, buffer.data.data());

    Game game{};
    init_game(game);

    double last_time = glfwGetTime();
    double accumulator = 0.0;
    const double dt = 1.0 / 60.0;

    while (!glfwWindowShouldClose(window)) {
        double now = glfwGetTime();
        accumulator += (now - last_time);
        last_time = now;

        while (accumulator >= dt) {
            update_game(game);
            accumulator -= dt;
        }

        render_game(buffer, game);

        glTexSubImage2D(GL_TEXTURE_2D, 0, 0, 0, buffer.width, buffer.height,
                       GL_RGBA, GL_UNSIGNED_BYTE, buffer.data.data());

        glClear(GL_COLOR_BUFFER_BIT);
        glDrawArrays(GL_TRIANGLES, 0, 3);

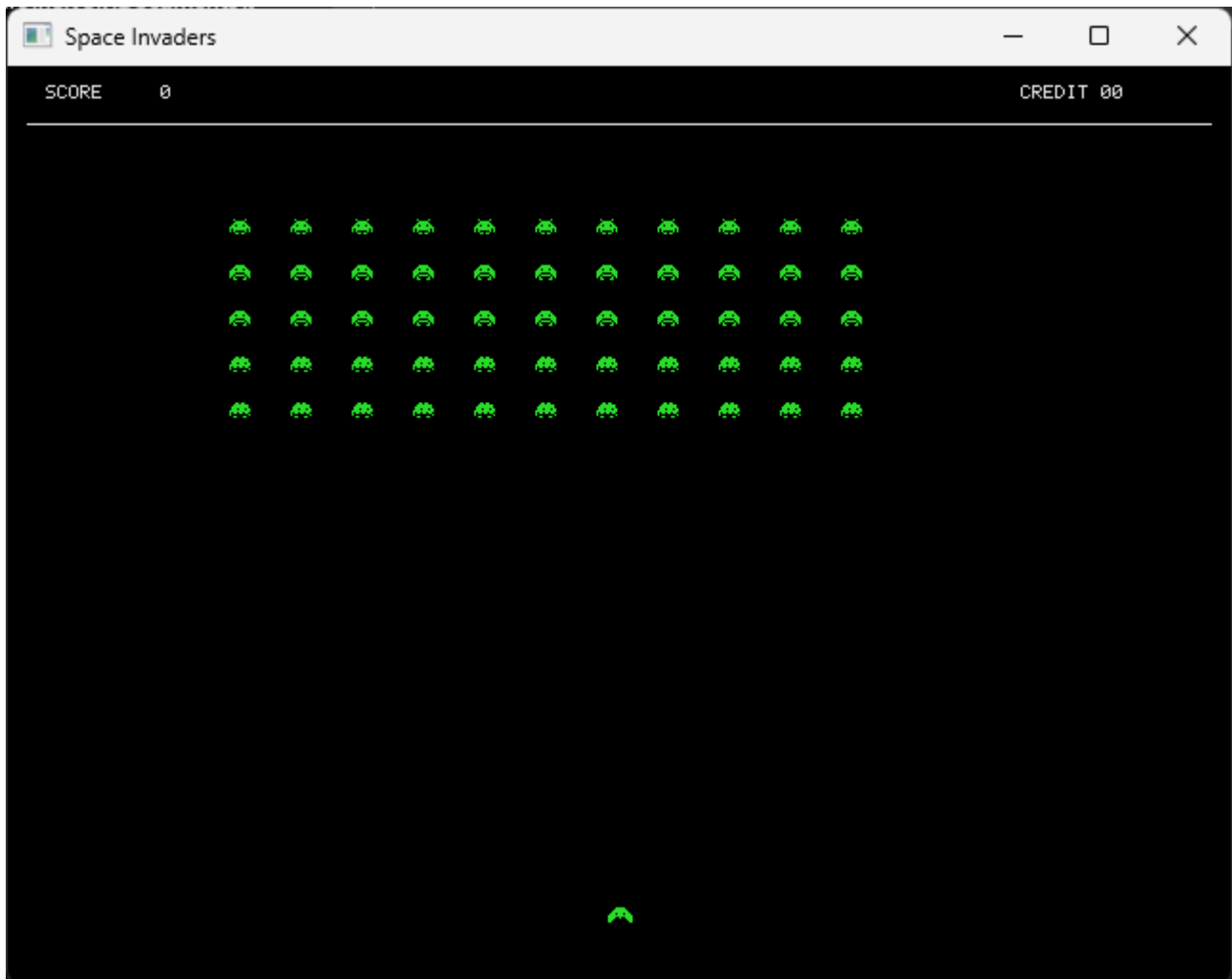
        glfwSwapBuffers(window);
        glfwPollEvents();
    }

```

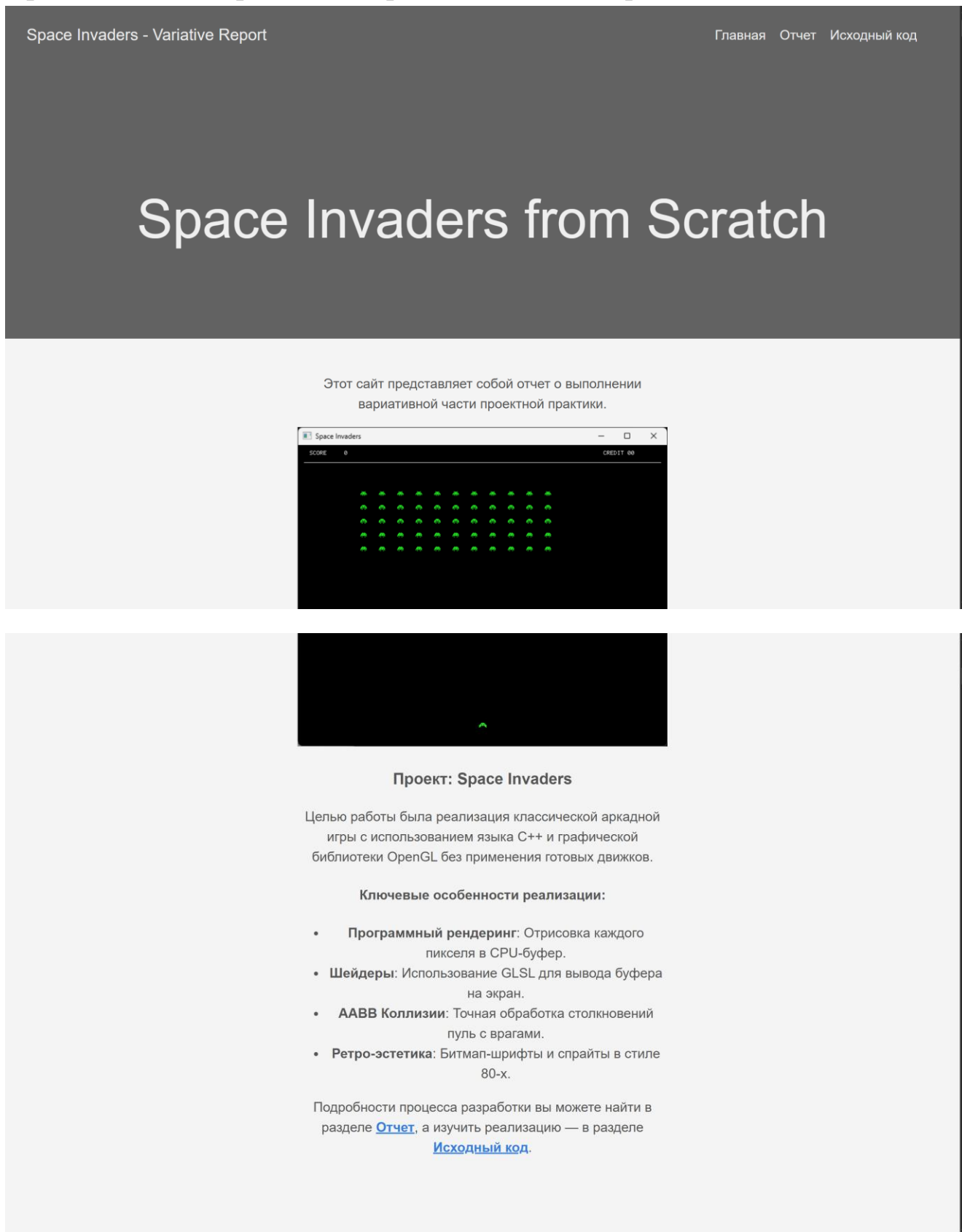


```
glDeleteTextures(1, &texture);  
glDeleteProgram(program);  
glDeleteVertexArrays(1, &vao);  
  
glfwDestroyWindow(window);  
glfwTerminate();  
return 0;  
}
```

Приложение Г. Скриншот In-Game



Приложение Д. Скриншоты страниц веб-сайта Вариативной части



SPACE INVADERS FROM SCRATCH

Отчет по вариативной части

Отчет по вариативной части практики: Реализация игры Space Invaders

1. Общая информация

Тема задания: Практическая реализация технологии «Build Your Own X» (из списка CodeCrafters). **Выбранный проект:** Создание игры Space Invaders «с нуля» на C++ и OpenGL. **Стек технологий:**

- **Язык:** C++ (стандарт C++17).
- **Компилятор:** g++ (MinGW-w64).
- **Система сборки:** CMake.
- **Библиотеки:** GLFW 3.4 (управление окнами и вводом), GLEW 2.3.1 (доступ к расширениям OpenGL).
- **ОС:** Windows.

На данном этапе была выполнена настройка библиотек GLFW и GLEW. Создано основное окно приложения размером 640x480 и настроен базовый игровой цикл.

Часть 2. CPU-буфер и графический конвейер

Реализован программный буфер отрисовки (массив пикселей `uint32_t`). Написаны вершинный и фрагментный шейдеры для вывода этого буфера как текстуры на «fullscreen-треугольник».

Часть 3. Игровые объекты и спрайты

Определены структуры для игрока, пришельцев и пуль. Создана система спрайтов на основе битмапов. Реализована сетка из 55 инопланетян с поддержкой анимации.

Часть 4. Интерактивность и коллизии

Добавлена обработка клавиш и механика стрельбы. Внедрена проверка столкновений по алгоритму AABB.

Часть 5. Пользовательский интерфейс и система очков

Разработан собственный шрифт формата 5x7 ASCII. Реализованы функции отрисовки текста и элементов UI (SCORE, надписи).

4. Результаты работы

В ходе выполнения была создана полностью функциональная игра Space Invaders с ретро-графикой, плавным геймплеем (60 FPS) и портативной

SPACE INVADERS FROM SCRATCH

Исходный код проекта

Ниже приведен полный исходный код игры **Space Invaders**, реализованный на C++ с использованием OpenGL (GLFW/GLEW).

```
#include <GL/glew.h>
#include <GLFW/glfw3.h>

#include <stdint>
#include <stdio>
#include <stdlib>
#include <string>
#include <algorithm>
#include <vector>

namespace {
constexpr int kBufferWidth = 640;
constexpr int kBufferHeight = 480;
constexpr int kAlienRows = 5;
constexpr int kAlienCols = 11;
constexpr int kAlienCount = kAlienRows * kAlienCols;
constexpr int kMaxBullets = 128;
constexpr int kPlayerSpeed = 3;
constexpr int kBulletSpeed = 4;
constexpr int kAlienAnimSpeed = 30;
```